Cuestionario / Actividad Práctica: Procesamiento en Tiempo Real con Spark y Kafka

Parte 1: Conceptos básicos – Responde a las siguientes preguntas:

Docker

- ¿Qué es un contenedor y cómo se diferencia de una máquina virtual?

Los contenedores son entornos de software ligeros, portátiles y aislados que empaquetan aplicaciones junto con sus dependencias para garantizar una ejecución consistente en diferentes plataformas. Simplifican el desarrollo, el despliegue y la administración, al tiempo que aseguran que las aplicaciones se ejecuten de manera confiable, independientemente de la infraestructura subyacente. La principal diferencia es que los contenedores comparten el kernel del sistema operativo anfitrión, mientras que las máquinas virtuales ejecutan un sistema operativo completo dentro de un entorno virtualizado.

- ¿Qué son los volúmenes en Docker y para qué sirven?

Los volúmenes de Docker son soluciones de almacenamiento persistente que se utilizan para administrar y almacenar datos fuera del sistema de archivos del contenedor, garantizando que los datos permanezcan intactos incluso si el contenedor se elimina o se vuelve a crear.

Son ideales para almacenar datos de aplicaciones, registros (logs) y archivos de configuración que deben conservarse después de reiniciar o actualizar los contenedores.

-Explica la diferencia entre docker run y docker compose.

La diferencia principal es que docker run permite crear y ejecutar un contenedor individual mediante comandos, mientras que docker compose permite definir y administrar varios contenedores como una aplicación mediante un archivo de configuración.

Docker run: Se utiliza para crear y ejecutar un contenedor directamente desde la terminal.

Docker compose: Se utiliza para definir y administrar uno o varios servicios en un archivo llamado compose.yaml o docker-compose.yml.

Kafka

Apache Kafka es una plataforma distribuida de streaming de eventos que permite capturar, almacenar, procesar y transmitir grandes volúmenes de datos en tiempo real entre diferentes aplicaciones y sistemas. Fue diseñada para manejar flujos continuos de eventos de manera escalable, tolerante a fallos y con alta disponibilidad.

-¿Qué es un topic en Kafka?

Categoría o flujo lógico donde se organizan los eventos.

¿Cuál es la función de Zookeeper dentro de un clúster Kafka?

Apache ZooKeeper es un servicio de coordinación distribuida que, en las arquitecturas tradicionales de Kafka, se utiliza para mantener metadatos del clúster, coordinar los brokers y ayudar a gestionar el estado del sistema.

Explica qué significan “particiones” y “replication factor” en un topic.

Particiones: permiten dividir un topic para distribuir los eventos entre varios brokers y procesarlos en paralelo.

Replication Factor: indica cuántas copias de cada partición se mantienen en el clúster.

¿Qué es Spark y cuál es su propósito?

Apache Spark es un motor de procesamiento de datos distribuido que permite procesar grandes volúmenes de información de forma rápida, tanto en memoria como en disco, utilizando varios núcleos o máquinas de un clúster. Su propósito es realizar tareas de procesamiento y análisis de Big Data, incluyendo transformaciones, consultas SQL, aprendizaje automático y procesamiento de flujos de datos.

Diferencia entre RDD, DataFrame y Dataset

- **RDD** (**Resilient Distributed Dataset**): colección distribuida de datos que ofrece control detallado sobre las operaciones, pero con menos optimización automática.

- **DataFrame**: colección distribuida de datos organizada en columnas, similar a una tabla de una base de datos. Permite optimizaciones automáticas y consultas SQL.

- **Dataset**: colección distribuida con tipos de datos definidos y comprobación de tipos en tiempo de compilación. Está disponible principalmente en Scala y Java.

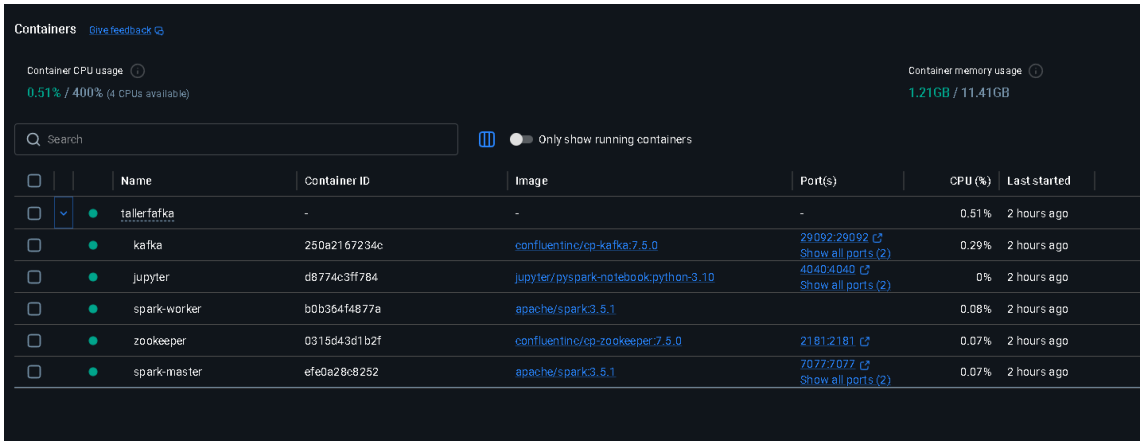
¿Qué es Spark Streaming y para qué se utiliza?

Spark Streaming permite procesar flujos continuos de datos en tiempo casi real, por ejemplo, eventos provenientes de Kafka, registros de aplicaciones o métricas de sensores. Se utiliza para analizar eventos conforme llegan, detectar anomalías, calcular métricas y generar alertas.

Parte 2: Actividad práctica con Docker + Kafka + Spark

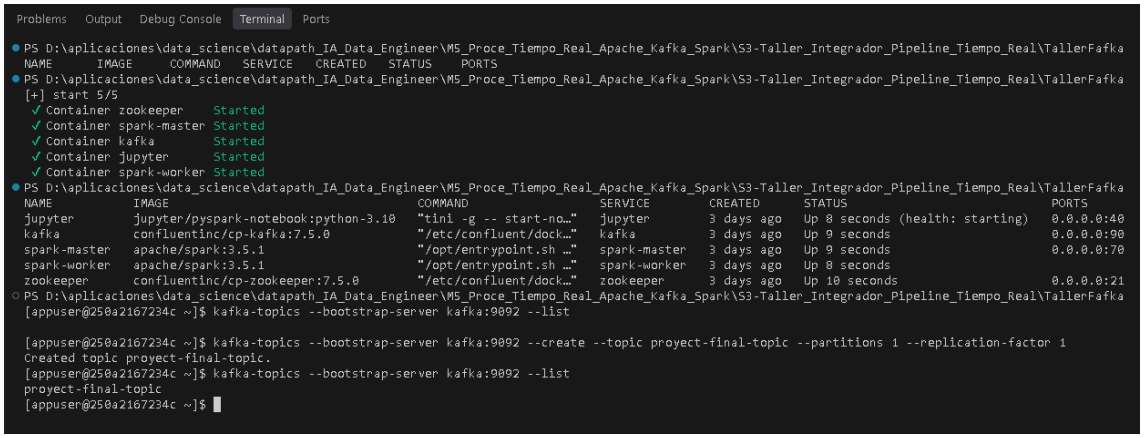
Ejercicio 1: Levantar los servicios

- Usando Docker Compose, crea un servicio de Kafka y otro de Zookeeper.
  Para esto utilizo el archivo .yml proveído en clase y levanto los servicios definidos.
- Verifica que ambos servicios estén corriendo.



Ejercicio 2: Crear un nuevo topic

- Crea un topic llamado actividad-topic con 1 partición y 1 réplica.
- Verifica que se haya creado correctamente.



Ejercicio 3: Enviar y leer mensajes

- Envía 5 mensajes simples al topic actividad-topic usando el producer de Kafka.
- Léelos usando el consumer y verifica que lleguen correctamente.

```python
# [1]: Importar librerías necesarias
from pyspark.sql import SparkSession
from pyspark.sql.functions import explode, split, col
from pyspark.sql.types import StringType

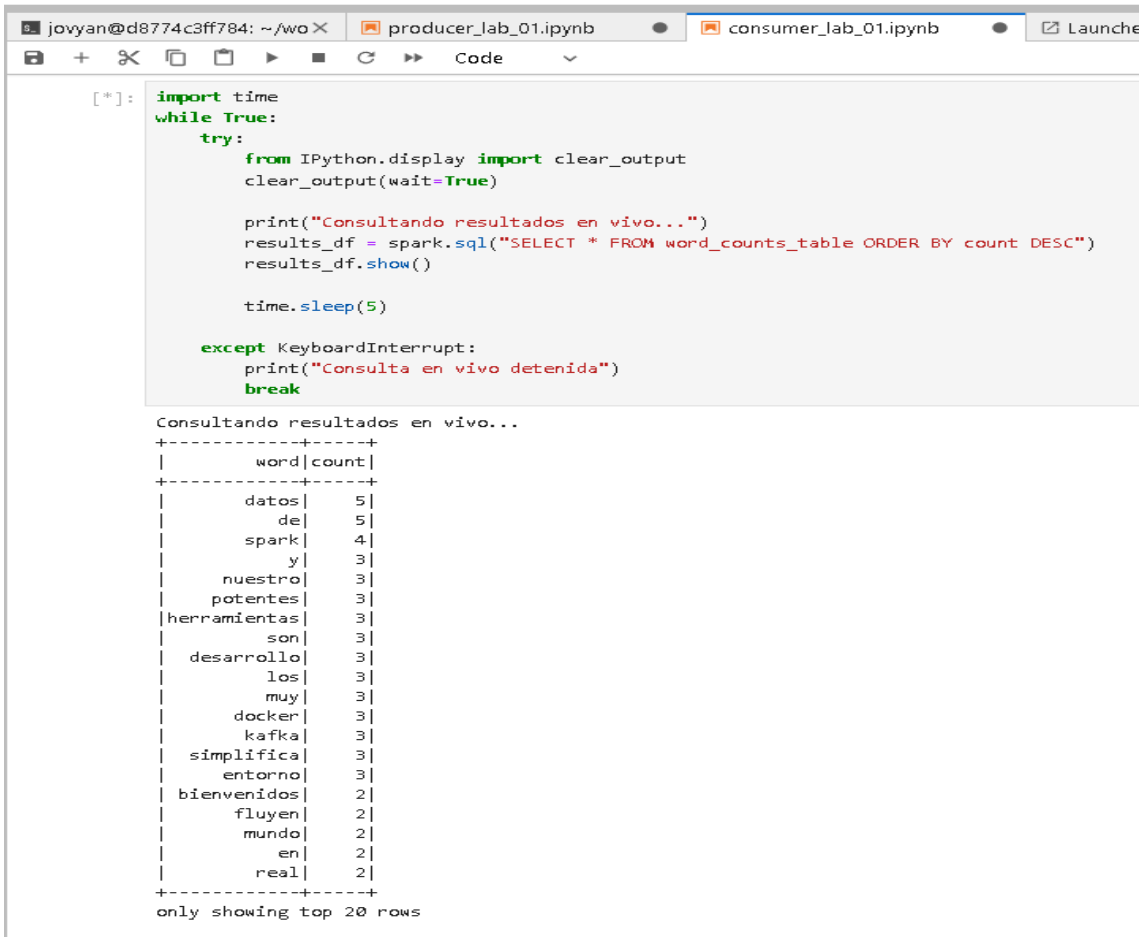
# [2]: Crear sesión de Spark
spark = SparkSession.builder \
    .appName("KafkaWordCount") \
    .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.0") \
    .getOrCreate()

# [3]: Leer stream desde Kafka
kafka_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe", "actividad-topic") \
    .load()
```

```python
kafka_df_2 = kafka_df.selectExpr("CAST(value as STRING)")

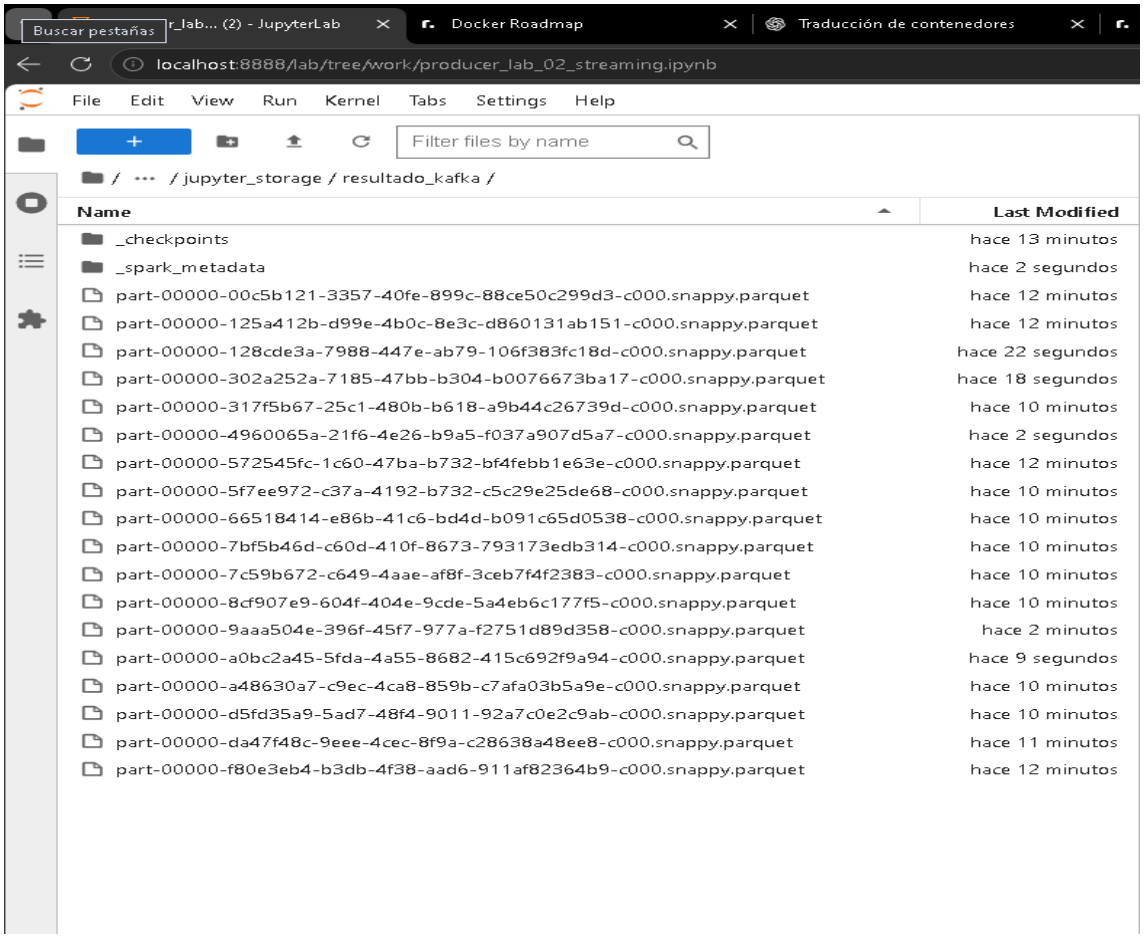
# Procesar el conteo de palabras
word_count_df = kafka_df_2 \
    .select(explode(split(col("value"), " ")).alias("word")) \
    .groupBy("word") \
    .count()

# Escribir el stream a memoria
query = word_count_df.writeStream \
    .outputMode("complete") \
    .format("memory") \
    .queryName("word_counts_table") \
    .start()
```



Ejercicio 4: Spark Streaming básico

- Crea un notebook o script de Spark que lea los mensajes de actividad-topic y los muestre en consola.



Ejercicio 5: Limpieza de recursos

- Elimina contenedores, imágenes y volúmenes creados durante la práctica para liberar espacio.

docker compose down -v --rmi local

docker rmi confluentinc/cp-zookeeper:7.5.0 confluentinc/cp-kafka:7.5.0 apache/spark:3.5.1 jupyter/pyspark-notebook:python-3.10

docker system prune -a --volumes -f

Parte 3: Comunicación de Apredizaje (link al artículo en Medium)

Cada alumno debe redactar un **artículo en Medium** contando su experiencia durante la actividad.

El artículo debe incluir: Lo que aprendieron sobre Docker, Kafka y Spark.

Los desafíos que enfrentaron y cómo los resolvieron.

Reflexiones sobre la importancia de procesar datos en tiempo real y el uso de estas herramientas en la práctica profesional.